

Document Object Model

connects web pages to scripts or programming languages by representing the structure of a document as a tree

The DOM represents the document as nodes and objects; so programming languages can interact with the page.

So we can have oop -> DOM methods and objects

*DOM API (application programming interface) -> set of methods and properties

An **interface** is a specification/contract describing the properties and methods available to a category of objects.

Each interface has its own methods and properties

Element -> id, getAttribute, classList

Node -> a general interface for all objects in a document -> appendChild,
removeChild

to access any element in an HTML page -> first access the document object.
The **document object** represents your **web page**.

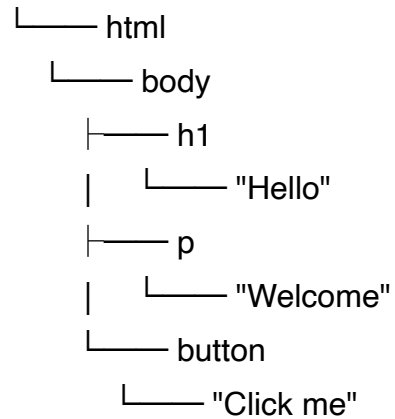
To manipulate HTML with JavaScript -> first **select an element**

using methods:
getElementById

... ByTagName, ...ByClassName, querySelector, querySelectorAll

A DOM tree :

Document



****Inline CSS:** (it's better to use classes instead)

modify css directly:

```
box.style.color = "red";
```

```
box.style.backgroundColor = "black"; ->(background-color becomes camelCase)
```

Navigating the DOM tree:

****To navigate around the DOM tree:**

parentNode, childNodes, firstChild/
lastChild, hasChildNodes(), getRootNode(), previousSibling/nextSibling.

****To navigate around the element tree:**

parentElement, children,
firstElementChild/lastElementChild, childElementCount, previousElementSibling/
nextElementSibling.

Adding elements:

append -> adds at the end,

appendChild -> adds as a child ,

prepend -> at the beginning ,

before -> before and outside the arg,

after -> after and outside

classList :

add, remove, toggle(adds if it isn't there, removes if it is)

##Events

*Event bubbling

event listener on the parent of a node -> will handle the events for children

currentTarget will be the node
you're listening on

<div> <button> </button> </div> -> adding an eventlistener for the
div element and then clicking the button will be an event for the div

event listeners on parents and the child -> event bubbles from children up to the
parents in the tree

body.addEventListener

div.addEventListener

button.addEventListener

#log :

button -> div -> body